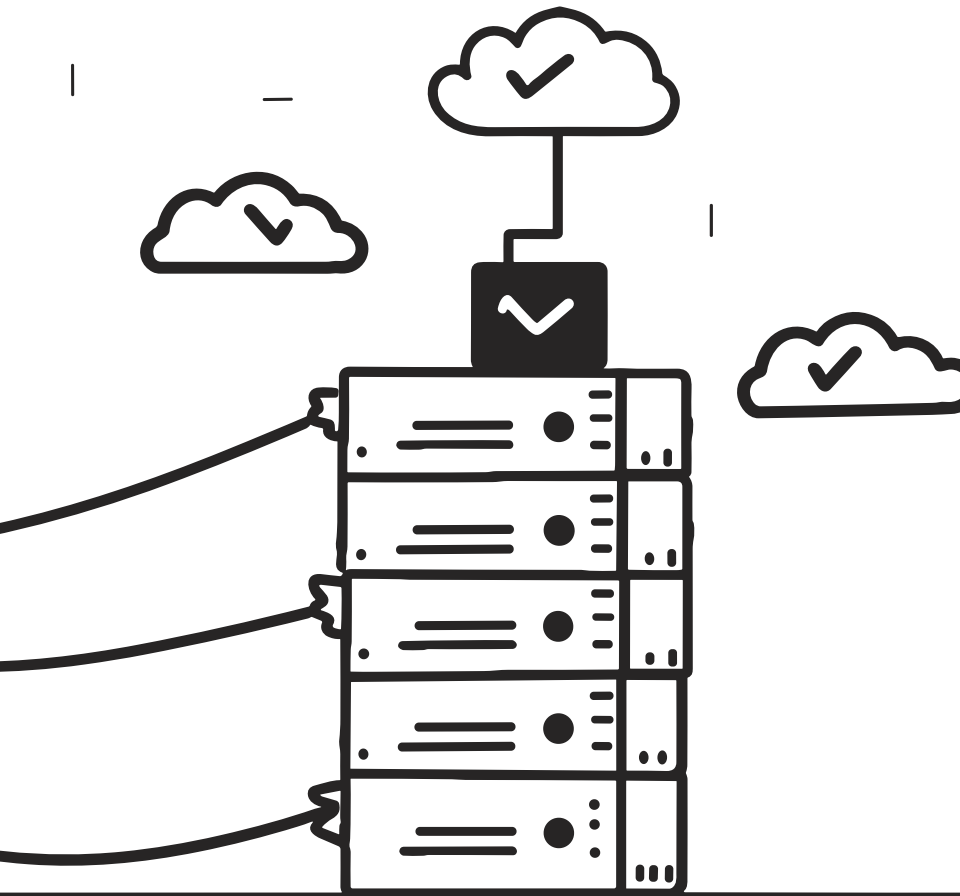




Teoria R2 — Formularis, petició POST i validació de servidor

Com un formulari HTML genera una petició real, com recollim les dades en PHP i per què la validació sempre ha de ser en servidor.

CICLE FORMATIU · DESENVOLUPAMENT WEB

Índex de la sessió

01

Del formulari al servidor

Com viatja una petició POST

02

Tipus de camps

Text, select/radio i checkbox

03

Lectura segura amb \$_POST

Accés robust a les dades

04

Validació en servidor

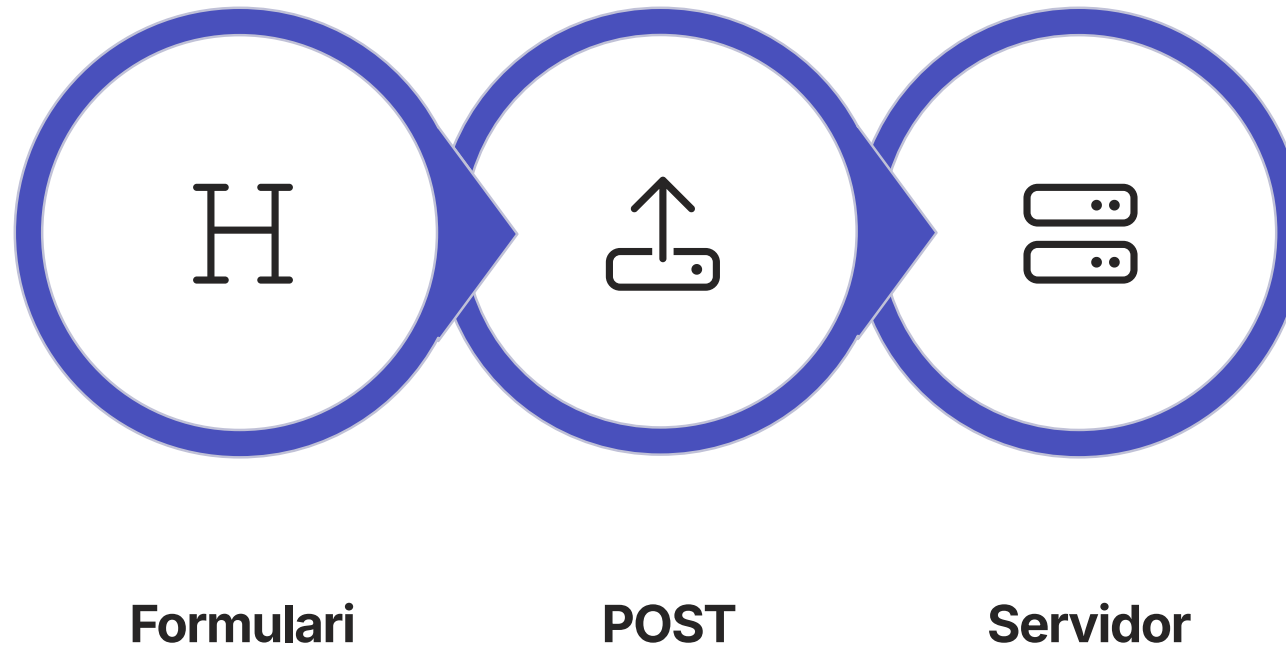
Per què no podem confiar en el client

05

Errors i resposta

Missatges útils i eixida escapada

Del formulari al servidor



Quan l'usuari fa clic a *Enviar*, el navegador empaqueta les dades del formulari i les envia al servidor com a petició HTTP POST. **method="post"** no valida res: únicament indica el mecanisme de transport.

Què arriba a \$_POST?

No tots els camps arriben igual. El tipus d'input condiona com (o si) apareix a la petició.

Text / Textarea

Sempre arriba, fins i tot si és buit. El valor és la cadena que ha escrit l'usuari.

```
$_POST['nom'] → "Anna"
```

Select / Radio

Arriba l'opció seleccionada. Si no n'hi ha cap marcada, pot no aparèixer.

```
$_POST['tipus'] → "urgent"
```

Checkbox

Si no està marcat, no apareix a \$_POST. Cal usar isset() per saber si existeix.

```
$_POST['urgent'] → absent!
```



El cas especial del checkbox

✓ Checkbox marcat

El navegador envia el camp amb el valor "on" (o el value definit). Apareix a `$_POST`.

```
isset($_POST['urgent']) →  
true
```

✗ Checkbox no marcat

El navegador **no envia res**. La clau directament no existeix a `$_POST`. Accedir-hi sense comprovar genera un avís.

```
isset($_POST['urgent']) →  
false
```

Lectura segura de les dades

Useu l'operador `??` (null coalescing) per evitar errors quan una clau no existeix, i `trim()` per netejar espais innecessaris.

```
$nom = trim($_POST['nom'] ?? "");  
$tipus = $_POST['tipus'] ?? "";  
$urgent = isset($_POST['urgent']);
```

- ❶ L'operador `??` retorna el valor de la dreta si la clau no existeix o és null. És l'alternativa segura a accedir directament a `$_POST['clau']`.

Validació en servidor

La validació HTML5 (`required`, `type="email"...`) és útil per a l'experiència d'usuari, però **no és segura**: qualsevol pot enviar una petició sense passar pel navegador.



Mai confieu únicament en la validació del client. La validació real sempre s'ha de fer en servidor.

```
$error = "";  
if ($nom === "") {  
    $error = 'El nom és obligatori';  
}
```

Una condició simple bloqueja el cas incorrecte i prepara un missatge d'error útil per a l'usuari.

Resposta amb error visible

Si hi ha error, el mostrem a la pàgina. Usem `htmlspecialchars()` per evitar que dades malicioses es renderitzin com a HTML.

```
<?php if ($error != ""): ?>
  <p class="error"><?= htmlspecialchars($error) ?></p>
<?php endif; ?>
```

❌ **XSS (Cross-Site Scripting):** Si mostres dades de l'usuari sense escapar, un atacant pot injectar codi JavaScript maliciós. `htmlspecialchars()` converteix caràcters perillosos en entitats HTML inofensives.

Errors habituals

Accés directe sense ??

`$_POST['nom']` sense `??` genera un *Notice* si el camp no existeix (p. ex. un checkbox no marcat).

Confiar en `method="post"`

`method="post"` no valida ni filtra res. Només descriu com viatgen les dades.

Mostrar dades sense escapar

Imprimir `$_POST['nom']` directament permet injeccions HTML/JS. Sempre cal `htmlspecialchars()`.

Oblidar camps buits

Un camp de text buit arriba com a cadena buida `""`, no com a `null`. Cal validar-ho explícitament.



Preguntes finals

- 1 Quines dades arriben a `$_POST` quan envies el formulari?
- 2 Què passa amb un checkbox si no està marcat?
- 3 Quina validació està realment en servidor?
- 4 Quin missatge veu l'usuari quan la dada no és vàlida?